

UrbanMove Smart Mobility on AWS

Cloud Computing Final Lab

Jordy Meye Huizhe Su Arihant Jain Nizar

EPITA – Master in Computer Science

April 2026

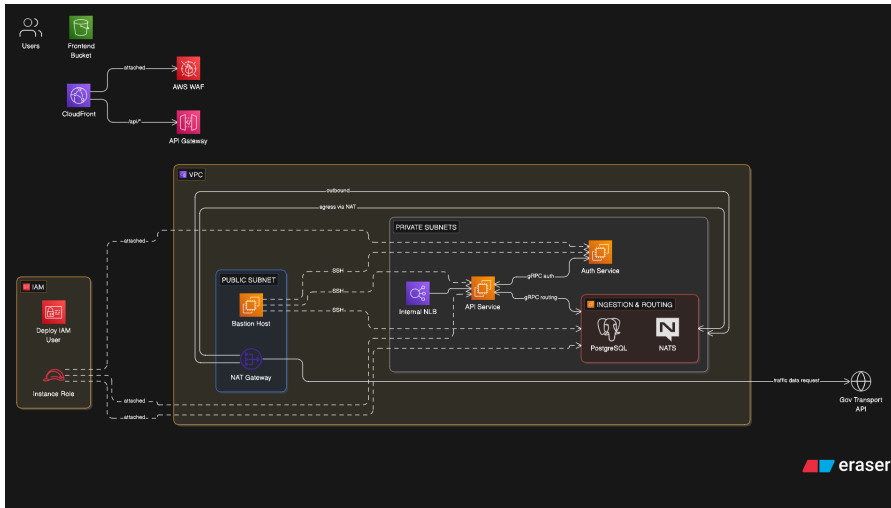
Agenda

- 1 Scope
- 2 Architecture
- 3 Security and Networking
- 4 Automation
- 5 Application Flow
- 6 Operations
- 7 Outcome

Project Scope

- Build and deploy a 3-service mobility platform.
- Keep service runtime in private subnets.
- Route API traffic through CloudFront and API Gateway.
- Add IAM baseline, WAF, monitoring, and cost controls.
- Deliver report, evidence, and demo-ready UI.

Final Architecture



Runtime Components

Component	Role
urbanmove-api	REST facade, request validation, calls auth and routing via gRPC
urbanmove-auth	JWT issue and validation, role checks
urbanmove-ingestion-routing	Gov feed ingestion, simulator fallback, route logic
PostgreSQL + NATS	Data persistence and messaging backbone
Bastion host	SSH jump host for private instances

Network and Security Controls

- CloudFront default origin: S3 frontend bucket.
- CloudFront /api/* origin: API Gateway.
- API Gateway private integration: VPC Link to internal NLB.
- Services run in private subnets; outbound only through NAT Gateway.
- WAF Web ACL associated with CloudFront distribution.
- IAM setup includes deploy identity and EC2 instance role/profile.

Infrastructure Evidence: Network

The screenshot displays the AWS Management Console for the VPC dashboard of vpc-0f2aa103e971372b0 / urbanmove-vpc. The interface includes a left-hand navigation menu with sections for Virtual private cloud, Security, and PrivateLink and Lattice. The main content area is divided into several panels: Details, DNS resolution, Main network ACL, IPv6 CIDR, Encryption control ID, State, Tenancy, DHCP options set, IPv4 CIDR, Route 53 Resolver DNS Firewall rule groups, DNS hostnames, Main route table, IPv6 pool, and Owner ID. A Resource map section at the bottom provides a visual overview of the VPC's components, including subnets (eu-west-3a, eu-west-3b), route tables, and network connections.

The screenshot displays the AWS Management Console for the VPC dashboard of nat-04e1fb8b44529f8d7 / urbanmove-natgw. The interface includes a left-hand navigation menu with sections for Virtual private cloud, Security, and PrivateLink and Lattice. The main content area is divided into several panels: Details, NAT gateway ID, NAT gateway ARN, VPC, Connectivity type, Primary public IPv4 address, State, State message, Primary private IPv4 address, Primary network interface ID, and Subnet. A Secondary IPv4 addresses section is also visible, showing a search bar and a table of addresses.

Infrastructure Evidence: Edge and Compute

Resources and protection packs (web ACLs)
View your secured assets and their active security controls.

Protection packs (web ACLs) (1) [Actions](#) [Create protection pack \(web ACL\)](#)

Use protection packs (web ACLs) to combine rules and rule sets that protect your infrastructure from threats.

Search protection packs (web ACLs) by: [Region scope](#) [CloudFront \(Global\) and Regional](#)

Name	Rules	Resources	Dashboard	AI traffic analysis	Sampled requests	Logging & metrics
urbanmove-cloudfront-waf	4 rules	Manage	View	View	View	Not enabled

Manage rules for urbanmove-cloudfront-waf

Rules are shown in ascending rule priority order.

- [AWS-IAM-ManagedRulesCommonRuleSet](#) 100,000
- [AWS-IAM-ManagedRulesBottingRuleSet](#) 100,000
- [AWS-IAM-ManagedRulesAmazonIPReputationList](#) 15,000
- [RateLimitRule](#) 2,000

[Add rule](#) [Cancel](#) [Edit rule order](#)

Instances (4) [Connect](#) [Instance state](#) [Actions](#) [Launch instances](#)

Save filter sets: [Choose filter set](#) [Find instance by attribute or tag \(case-sensitive\)](#)

[Instance state = running](#) [Clear filters](#)

Name	Instance ID	Instance state	Instance type	States check	Alarm status	Availability Zone	Public IPv4 DNS	Public IPv6
urbanmove-ec2-1	i-046c1b4a6e45354	Running	t3.micro	5/5 checks passed	View alarms	eu-west-1a	-	-
urbanmove-ec2-2	i-01923b6c79a1a12	Running	t3.micro	5/5 checks passed	View alarms	eu-west-1a	-	-
urbanmove-ec2-3	i-04567f1c07270437c	Running	t3.micro	5/5 checks passed	View alarms	eu-west-1a	ec2-15-238-95-104.eu...	15.238.95
urbanmove-ec2-4	i-04a09f1b1723b5159	Running	t3.micro	5/5 checks passed	View alarms	eu-west-1a	-	-

Select an Instance

Infra Provisioning Script Sequence

```
cd lab_final/infra/aws-cli
./01_budget_and_tags.sh
./01a_identity_iam_setup.sh
./02a_create_vpc_private_network.sh
./02_create_ec2_topology.sh
./03a_api_gateway_setup.sh
./03_s3_cloudfront_setup.sh
./04_cloudwatch_setup.sh
./06_waf_setup.sh
```

Deployment Script Sequence

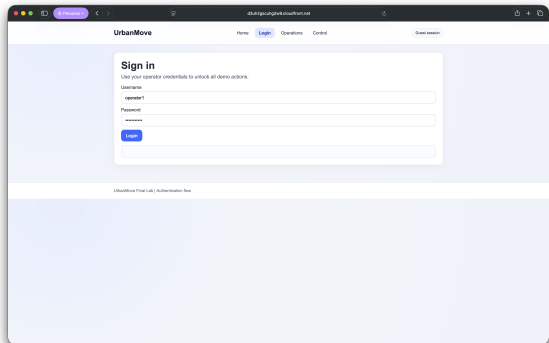
```
cd lab_final/infra/deploy
./01_bootstrap_hosts.sh
./02_deploy_auth.sh
./03_deploy_ingest.sh
./04_deploy_api.sh
./05_upload_frontend.sh
./06_smoke_test.sh
```

API Flow Evidence

The screenshot displays the AWS API Gateway console interface. The browser window at the top shows the URL `eu-west-3.console.aws.amazon.com` and the date `Sat 18 Apr 10:33`. The console navigation pane on the left includes sections for **API Gateway** (APIs, Custom domain names, Domain name access associations, VPC links), **Develop** (Routes, Authorization, Integrations, CORS, Reimport, Export), **Deploy** (Stages), **Monitor** (Metrics, Logging), and **Protect** (Throttling). The main content area is titled **Routes** for the API `urbanmove-routing-http-api (nwtwjp4fb)`. It features a **Create** button and a search bar. A tree view of routes is shown, with `/api` expanded to reveal `/api/proxy*` (selected), `/gov-feed`, `/health`, `/ingest`, `/auth`, and `/api`. The **Route details** panel on the right shows the route `ANY /api/proxy* (ID: c85q3i8)` with its **ARN** `arn:aws:apigateway:eu-west-3:/apis/nwtwjp4fb/routes/c85q3i8`. It includes sections for **Authorization** (no authorizer attached), **Integration** (backend resource `ucluf8b`), and buttons for **Delete**, **Edit**, **Attach authorization**, and **Configure**. The footer contains links for **CloudShell**, **Feedback**, **Console Mobile App**, and copyright information for Amazon Web Services, Inc.

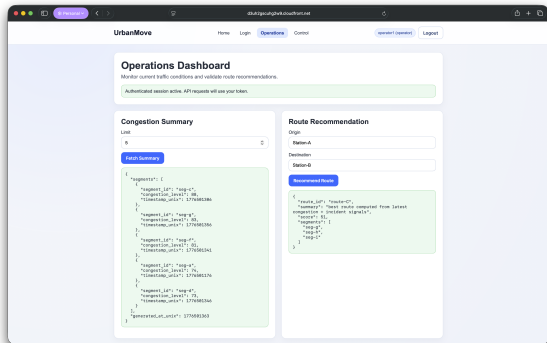
Frontend Flow

Login page



The screenshot shows the UrbanMove web application's login page. The browser's address bar displays 'd3a2g9u4g9k8.douchfront.net'. The page has a light blue header with the 'UrbanMove' logo and navigation links: 'Home', 'Login' (active), 'Operations', and 'Control'. A 'Guest session' button is in the top right. The main content area features a 'Sign in' section with the instruction 'Use your operator credentials to unlock all demo actions.' Below this are two input fields: 'Username' (containing 'operator1') and 'Password' (with masked characters). A blue 'Login' button is positioned below the password field. At the bottom of the page, a small text link reads 'UrbanMove Front Lib | Authentication Box'.

Operations and control pages



The screenshot displays the UrbanMove 'Operations Dashboard'. The browser's address bar shows 'd3a2g9u4g9k8.douchfront.net'. The header is identical to the login page, but the 'Operations' link is now active. The dashboard includes a green status bar at the top stating 'Authenticated session active. API requests will use your token.' Below this, there are two main panels. The left panel, titled 'Congestion Summary', has a 'Limit' dropdown set to '0' and a blue 'Fetch Summary' button. It contains a JSON object with an array of 'segments'. The right panel, titled 'Route Recommendation', has input fields for 'Origin' (Station-A) and 'Destination' (Station-B), a blue 'Recommend Route' button, and a JSON object with route details. The JSON data in both panels is as follows:

```
{
  "segments": [
    {
      "segment_id": "seg-a",
      "congestion_level": 88,
      "timestamp_unix": 1776961386
    },
    {
      "segment_id": "seg-g",
      "congestion_level": 89,
      "timestamp_unix": 1776961386
    },
    {
      "segment_id": "seg-f",
      "congestion_level": 85,
      "timestamp_unix": 1776961341
    },
    {
      "segment_id": "seg-a",
      "congestion_level": 74,
      "timestamp_unix": 1776961376
    },
    {
      "segment_id": "seg-a",
      "congestion_level": 73,
      "timestamp_unix": 1776961346
    }
  ],
  "generated_at_unix": 1776961363
}
```

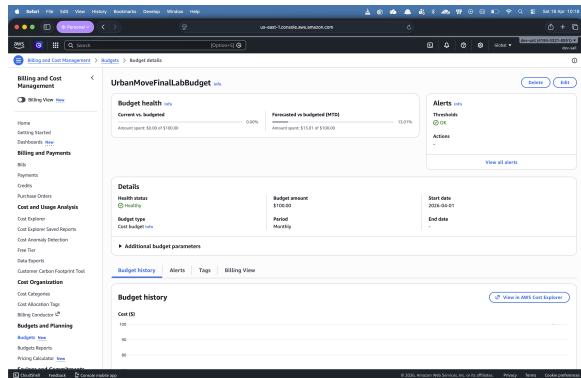
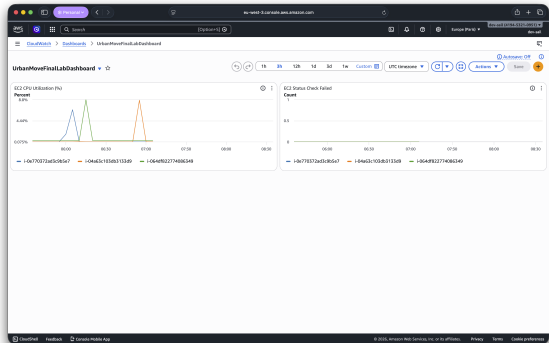
The 'Route Recommendation' JSON object contains:

```
{
  "route_id": "route-C",
  "summary": "Best route computed from latest congestion & incident signals",
  "score": 81,
  "segments": [
    "seg-g",
    "seg-a",
    "seg-l"
  ]
}
```

End-to-End Validation

[illegible]

Monitoring and Budget Evidence



Backup and Restore Drill (Non-Destructive)

```
# on ingest host
sudo docker exec urbanmove-postgres \
    pg_dump -U urbanmove -d urbanmove > /tmp/urbanmove_backup.sql

sudo docker exec -i urbanmove-postgres psql -U urbanmove -d postgres \
    -c "DROP DATABASE IF EXISTS urbanmove_restore;"

sudo docker exec -i urbanmove-postgres psql -U urbanmove -d postgres \
    -c "CREATE DATABASE urbanmove_restore;"

cat /tmp/urbanmove_backup.sql | sudo docker exec -i urbanmove-postgres \
    psql -U urbanmove -d urbanmove_restore
```

Sample verification output: relation list returned from urbanmove_restore with expected tables.

Issues Resolved During Implementation

- API route mismatch causing 404 from CloudFront path handling.
- API Gateway integration mapping updates for backend path rewrite.
- Ingestion host dependency issues causing 502.
- Private deployment SSH proxy reliability through bastion.
- VPC Link subnet/AZ fallback handling and NLB behavior tuning.

- Private VPC architecture with NAT egress and bastion access.
- CloudFront + WAF + API Gateway private integration.
- Multi-page frontend UI for demo flow.
- Final report with evidence: `lab_final/report/lab_report_final.pdf`
- Presentation source: `lab_final/report/final_presentation.tex`

Questions